Programación Concurrente y Paralela

Departamento de Computación – FCEyN – UBA

Segundo cuatrimestre de 2026

Primer parcial – 2do. cuatrimestre de 2026

Nombre y apellido: _____

Nº orden: _____ L.U.: _____ Cant. hojas: ___

1	2	3	4	Nota

- Cada ejercicio debe realizarse en hojas separadas y numeradas. Debe identificarse cada hoja con nombre, apellido, LU y número de orden. Hojas sin identificar no serán corregidas.
- Entregue esta hoja junto al examen, la misma **no** se incluye en la cantidad total de hojas entregadas.
- **Cada código o pseudocódigo debe estar bien explicado y justificado. ¡Obligatorio!**
- Toda suposición o decisión que tome deberá justificarla adecuadamente. Si la misma no es correcta o no se condice con el enunciado no será tomada como válida y será corregida acorde.
- La devolución de los exámenes corregidos es personal. Los pedidos de revisión se realizarán por escrito, antes de retirar el examen corregido del aula.
- Los parciales tienen dos notas: I (Insuficiente) y A (Aprobado).
- Para tener la nota aprobada es necesario tener al menos dos ejercicios calificados como bien y el restante al menos regular. Además, alguno de los ejercicios calificados como bien debe ser el de monitores o el de semáforos.

Ejercicio 1.

Considere la siguiente propuesta para resolver el problema de la exclusión mutua para N threads con identificadores distintos $0, \ldots, N-1$:

```
1  global int turnos[N] = {-1, ..., -1}
2  global int atendiendo = 0, siguienteTurno = 0
3
4  int pedirTurno(int id) {
5     int pos = siguienteTurno
6     turnos[siguienteTurno] = id
7     siguienteTurno = (siguienteTurno + 1) % N
8     return pos
9  }
```

```
10 thread(id) {
11    while (true) {
12       seccion_no_critica()
13       pedirTurno(id)
14       while (turnos[atendiendo] != id) {}
15       seccion_critica()
16       atendiendo = (atendiendo + 1) % N
17    }
18 }
```

a) Asumiendo que sólo son atómicas la lectura y la escritura de variables, justifique adecuadamente que esta propuesta **no** resuelve el problema de la exclusión mutua.

b) Asumiendo ahora que `pedirTurno` **sí** es atómica, determine si esta propuesta resuelve el problema de la exclusión mutua. Justifique cada propiedad por separado, se cumpla o no.

c) Analice el caso con este código (con `pedirTurno` atómica) donde llega un $(N+1)$-ésimo thread con `id` N. ¿Qué propiedad no se cumple? Muéstrelo.

Ejercicio 2.

Julieta compró el centro de esquí del Cerro Chapelco, en San Martín de los Andes, y reemplazó las viejas cabinas "huevitos" por una telecabina moderna. Como no quiere pasarse la temporada indicando a mano quién sube a cada cabina, decidió automatizar la operación, empezando por una simulación.

La telecabina tiene M cabinas colgadas de un mismo cable, cada una con capacidad para C personas. Empiezan todas en fila, ordenadas por id de forma ascendente. Es decir, la cabina de id 0 empieza primera, la de id 1 segunda y asi.

Abajo hay una única estación de embarque, por la que pasa una cabina por vez. En esta primera versión de la simulación se asume que ninguna cabina parte con un lugar vacío, así que la cabina abre sus puertas y espera hasta completar sus C pasajeros. Como todas las cabinas cuelgan del mismo cable, en un modelado correcto nunca una cabina debería adelantarse a otra, es decir, llegan arriba en el mismo orden en que salieron.

Arriba hay una única estación de descenso. Una cabina no puede abrir sus puertas ahí hasta que la anterior se haya vaciado por completo. Los esquiadores que descienden bajan por la pista, vuelven a la estación de abajo y se ponen en la fila otra vez. Además, a lo largo del día pueden llegar nuevos esquiadores, que se suman a la fila de la estación de abajo.

Se pide modelar la simulación con un thread por cabina y un thread por cada esquiador, que se crea cuando el esquiador llega al centro. Cada cabina puede averiguar su propio identificador, un número entre 0 y $M-1$, invocando `idCabina()`. El esquiador ejecuta `subir()`, que le indica en qué cabina le tocó viajar, y luego `bajar(cabina)`. Tome en cuenta que las acciones de entrar y salir de las estaciones de embarque y decenso, así como

las acciones de subir y bajar de las cabinas, toman tiempo que deben ser respetados. Puede asumir que en todo momento hay al menos C esquiadores en el centro.

a) Proponga, en pseudocódigo, una solución usando semáforos que resuelva el problema. Puede usar el constructor `Semaphore(n)` y las primitivas `acquire()` y `release()`. Salvo que se indique lo contrario, los semáforos se asumen débiles. La solución no debe tener deadlocks ni race conditions. Justifique todas las decisiones tomadas.

b) Indique si su solución puede sufrir starvation. Si no puede, justifique por qué. Si puede, describa una traza que lleve a esa situación.

Ejercicio 3.

Victoria es fanática de los autos y tiene un taller en su casa donde arma, siempre a escala, un mismo modelo de auto parecido a los de una famosa marca italiana. Cada unidad se completa combinando 2 faros y 1 carrocería. Varios colaboradores le acercan piezas de a una, siempre del mismo tipo (cada colaborador siempre trae faros, o siempre trae carrocerías, nunca de los dos).

Victoria tiene un único banco de trabajo, y en él junta las piezas de una sola unidad por vez. Por eso no recibe piezas que le sobren para la unidad que está juntando. Un colaborador que llega con un faro cuando en el banco ya hay 2 faros tiene que esperar con la pieza en la mano hasta que empiece la unidad siguiente, y lo mismo le pasa al que trae una carrocería cuando en el banco ya hay una.

Cuando en el banco queda el juego completo de piezas (2 faros y 1 carrocería), Victoria arma la unidad, cosa que le lleva un buen rato. Al terminarla, despeja el banco y empieza a juntar las piezas de la unidad siguiente. Los colaboradores que aportaron las piezas de una unidad se quedan esperando juntos hasta que Victoria termina de armarla, porque quieren verla lista antes de volver a buscar la próxima pieza.

Modele la simulación con un thread por colaborador y un thread para Victoria. Cada colaborador invoca `traerFaro()` o `traerCarroceria()` cada vez que trae una pieza, y esa invocación no retorna hasta que la unidad a la que contribuyó esté terminada. Victoria, por su lado, repite indefinidamente el ciclo `esperarPiezas()`, armar la unidad, `terminarUnidad()`. La primera no retorna hasta que en el banco esté el juego completo, y la segunda da la unidad por terminada y despeja el banco. El armado en sí no hace falta modelarlo, pero tenga en cuenta que le lleva tiempo, así que Victoria no puede quedarse con el monitor tomado mientras arma.

Puede asumir que a lo largo de toda la simulación siguen llegando colaboradores de los dos tipos.

a) Resuelva, en pseudocódigo, usando monitores con semántica de Mesa. Las colas de condición no son fair y no hay *spurious wakeups*. La solución no debe tener deadlocks ni race conditions, y debe apuntar a minimizar la cantidad de threads que son despertados aunque aún no puedan completar su ejecución. Justifique todas las decisiones tomadas. ¿Tienen su solución Starvation?

b) Si tuviera que resolver esto con semántica de Hoare, ¿qué cambios tendría que hacer y por qué? Explique con palabras, no es necesario dar el código.

Ejercicio 4.

Considere la siguiente clase que representa una cuenta bancaria.

```
1  class Cuenta {
2      private int depositos=0;
3      private int extracciones=0;
4
5      public void deposito(int monto) {
6          depositos += monto;
7      }
8
9      public int saldo() {
10         return depositos - extracciones;
11     }
12     public boolean extraccion(int monto) {
13         if (depositos - extracciones < monto)
           {
14             return false;
15         }
16         extracciones += monto;
17         return true;
18     }
19 }
```

a) Dar una solución usando *locks* de granularidad fina que permita la ejecución concurrente de un depósito y una extracción. Justificar la correctitud de su solución (en términos de linearizabilidad).

b) Analizar el progreso del programa (presencia/ausencia de deadlocks, presencia/ausencia de starvation).

c) Mostrar que si se implementa `saldos()` libre de bloqueos, es decir, `public int saldo() { return depositos - extracciones; }`, la solución no es correcta (linearizable).

d) Dar una solución *lock-free* para el problema, y justificar su correctitud (en términos de linearizabilidad).

e) Indicar si en su solución *lock-free* el método `extraccion()` es *wait-free*. Justificar.